Cross-Site Scripting basado en DOM (DOM XSS) mediante `innerHTML`

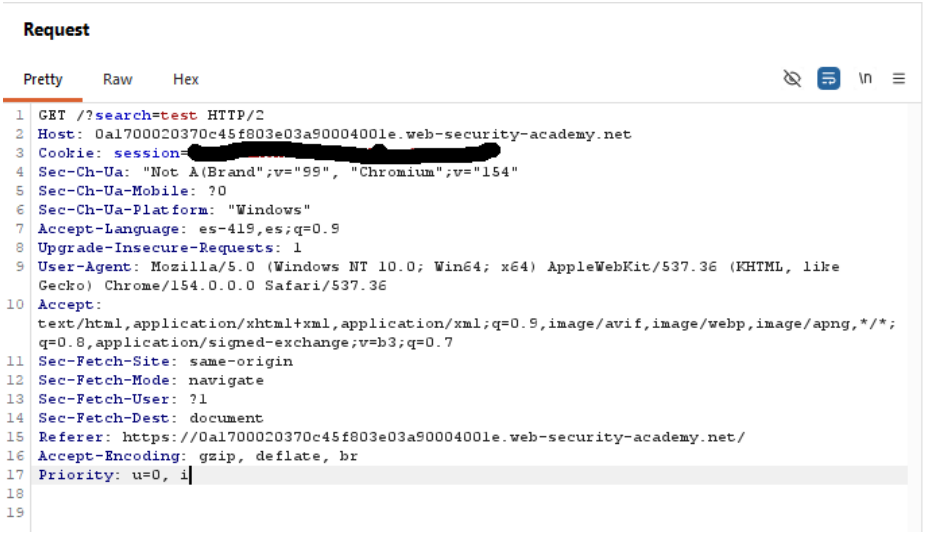
1. Descripción

La aplicación presentaba una vulnerabilidad de Cross-Site Scripting basado en DOM (DOM XSS). El código JavaScript del lado del cliente obtenía el valor del parámetro `search` de la URL y lo insertaba en la página mediante la propiedad `innerHTML`, sin una sanitización adecuada.

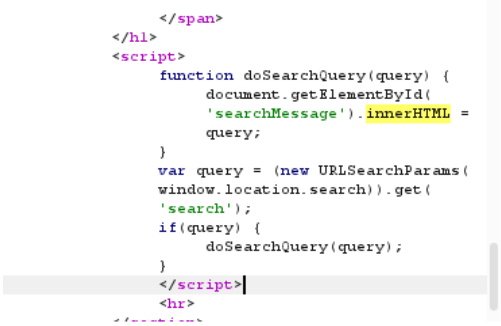
Como la entrada se interpretaba como HTML en lugar de texto plano, fue posible introducir un payload que provocó la ejecución de JavaScript en el navegador.

2. Evidencia

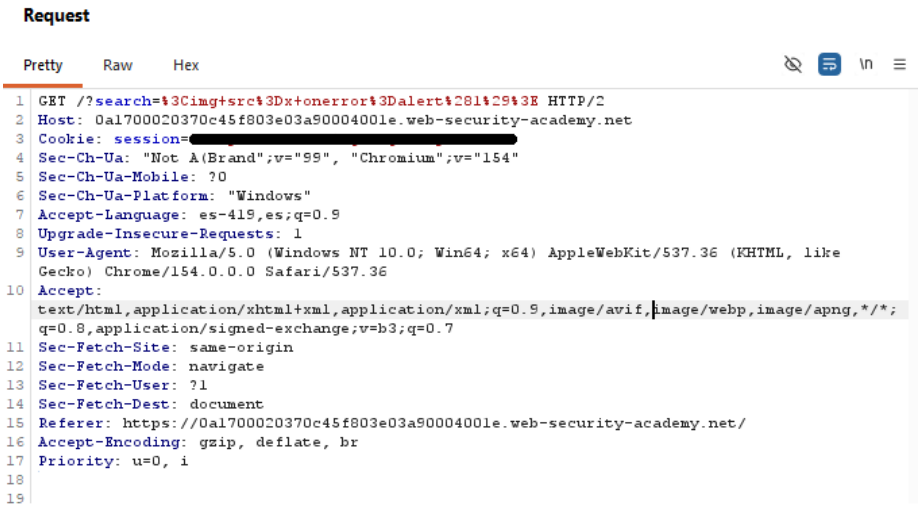
Se recopilaron las siguientes evidencias durante la prueba:

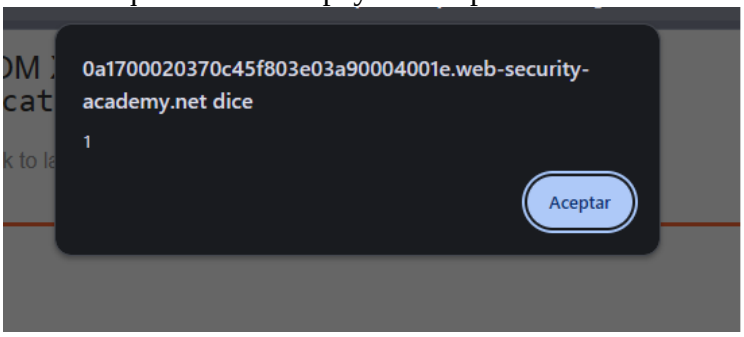


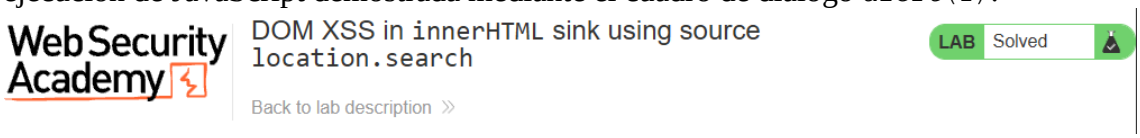
- solicitud de búsqueda inicial mediante `/?search=test`.



- código JavaScript que muestra el flujo del dato desde `URLSearchParams` hasta el uso de `innerHTML`.

- 

- solicitud que contiene el payload de prueba.
- 

- ejecución de JavaScript demostrada mediante el cuadro de diálogo `alert(1)`.
- 

confirmación de que el laboratorio de PortSwigger fue resuelto.

3. Pasos para reproducir

1. Acceder al laboratorio y realizar una búsqueda.
2. Inspeccionar el código JavaScript del lado del cliente e identificar el parámetro `search` de la URL.
3. Comprobar que el valor obtenido se pasa a `document.getElementById('searchMessage').innerHTML`.
4. Enviar el siguiente payload como valor del parámetro de búsqueda:

   ```
   <img src=x onerror=alert(1)>
   ```

5. Observar la aparición del cuadro de diálogo JavaScript, lo que confirma la ejecución de código mediante DOM XSS.

4. Impacto

Confidencialidad: potencialmente afectada. Según el contexto de la aplicación y los controles del navegador, una vulnerabilidad XSS podría permitir acceder a información disponible en el contexto de la víctima.

Integridad: potencialmente afectada. La ejecución de código en el navegador podría permitir realizar acciones en el contexto de la víctima, dependiendo de los controles de seguridad de la aplicación.

Disponibilidad: potencialmente afectada en determinados escenarios, por ejemplo, si el código malicioso interfiere con el funcionamiento de la aplicación. Este impacto no fue demostrado durante la prueba.

El resultado confirmado fue la ejecución de JavaScript en el navegador. No se verificó de forma independiente el acceso a información sensible, la realización de acciones no autorizadas ni la interrupción del servicio.

5. Causa raíz

La aplicación insertaba un valor controlado mediante la URL en un punto de inserción que interpreta HTML (`innerHTML`), sin una validación, sanitización o manipulación segura del DOM adecuada.

6. Remediación

- Evitar la inserción de entradas no confiables mediante `innerHTML`.
- Utilizar `textContent` cuando el objetivo sea mostrar datos proporcionados por el usuario como texto.
- Si es necesario representar HTML, emplear una biblioteca de sanitización mantenida y configurada con una lista permitida de elementos y atributos.
- Aplicar codificación de salida adecuada al contexto y validación de entradas cuando corresponda.
- Incorporar pruebas de seguridad que verifiquen que los parámetros de la URL no puedan provocar la ejecución de scripts.

7. Severidad y CVSS

Severidad provisional: pendiente de evaluación según el contexto real de la aplicación y las condiciones de explotación.

No corresponde asignar un puntaje CVSS definitivo basándose únicamente en la ejecución de `alert(1)`. La puntuación depende de factores como el vector de ataque, los privilegios requeridos, la interacción de la víctima, el alcance y el impacto demostrado.

8. Alcance de la prueba

La vulnerabilidad fue reproducida en un laboratorio educativo de PortSwigger Web Security Academy. Las pruebas se limitaron al entorno del laboratorio.